CompSci 390-03-26S

Distributed and Networked Systems

Workflows and Stragglers

I borrowed a bunch of slides from Bruce Maggs, who borrowed some of them from various sources: Danyang Zhuo, me, Jeff Dean@Google, other authors of the works discussed, MIT EECS 6.824.

Overview

- We have focused on **services** in this course.
  - Let's talk about stragglers: Tail at Scale
- Distributed systems are also for **data-intensive computation**:
  - Parallel → speedup and high throughput
  - But must tolerate failures!
- Today we touch on a progression of workflow systems:
  - MapReduce→Spark→TensorFlow and others
- Themes:
  - Abstractions and their tradeoffs
  - Evolution of abstractions for ML training and accelerators

Tail at Scale

- It's all about tail latency
- SLOs focus on tail latency
- Servers have variable latency (response times): why?
- What approaches to reduce the problem, for services?

DOI:10.1145/2408776.2408794

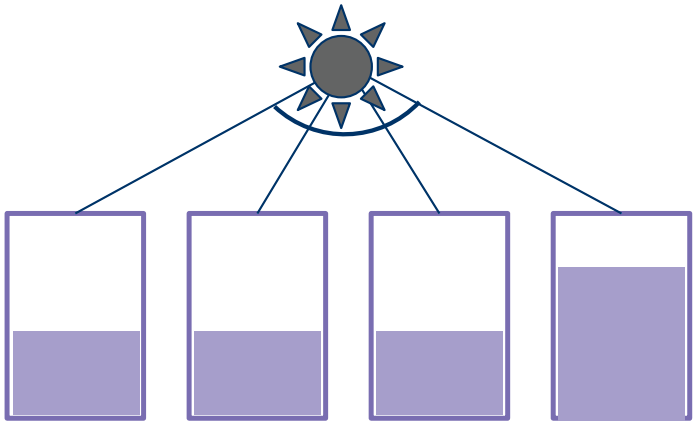
Software techniques that tolerate latency variability are vital to building responsive large-scale Web services.

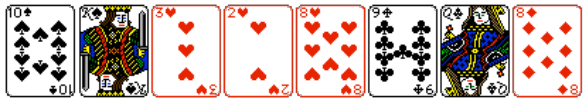
BY JEFFREY DEAN AND LUIZ ANDRÉ BARROSO

The Tail at Scale

Communications of the acm
february 2013 | vol. 56 | no. 2

Sharding and load balancing

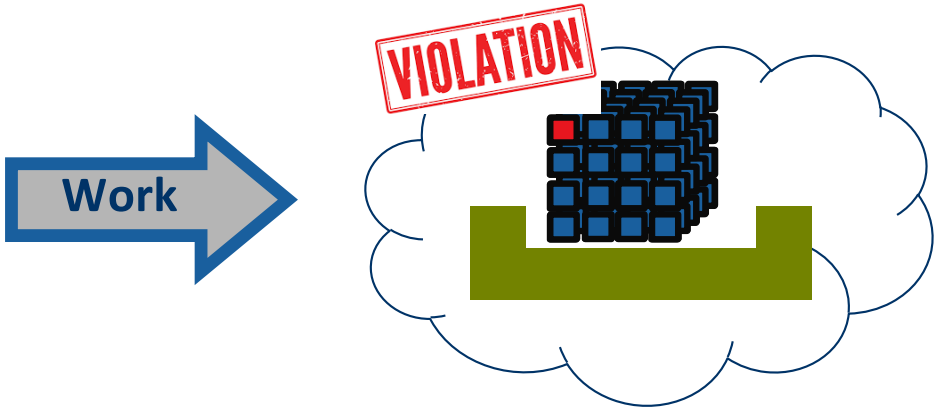




Means are misleading! **Mind the quantiles**. Example SLO: "99% of requests complete in 10ms or less".

Requests that land on the hot spot experience higher response times on average.

A hot spot limits throughput and/or may increase response time for some requests: the **tail** of the response time distribution.



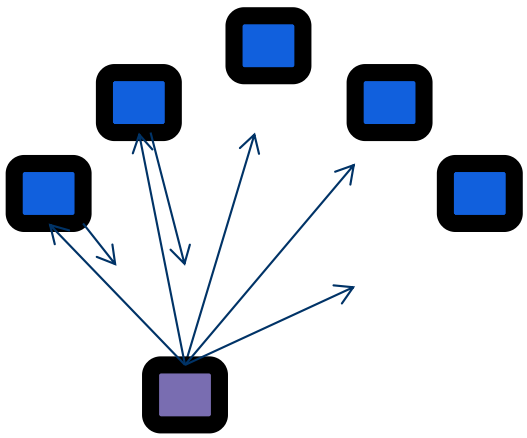
Tail at Scale: adaptations for requests

- **Tail-tolerant** services: How to live with latency variability?
- A: send request to multiple servers, take the first reply.
- Costs?
- "**Within-request short-term adaptations**"
  - Apply to individual requests
  - Especially applicable to services with **fanout**: each request results in N>1 requests sent (in parallel) to backends? (e.g., search)
  - Fanout examples?

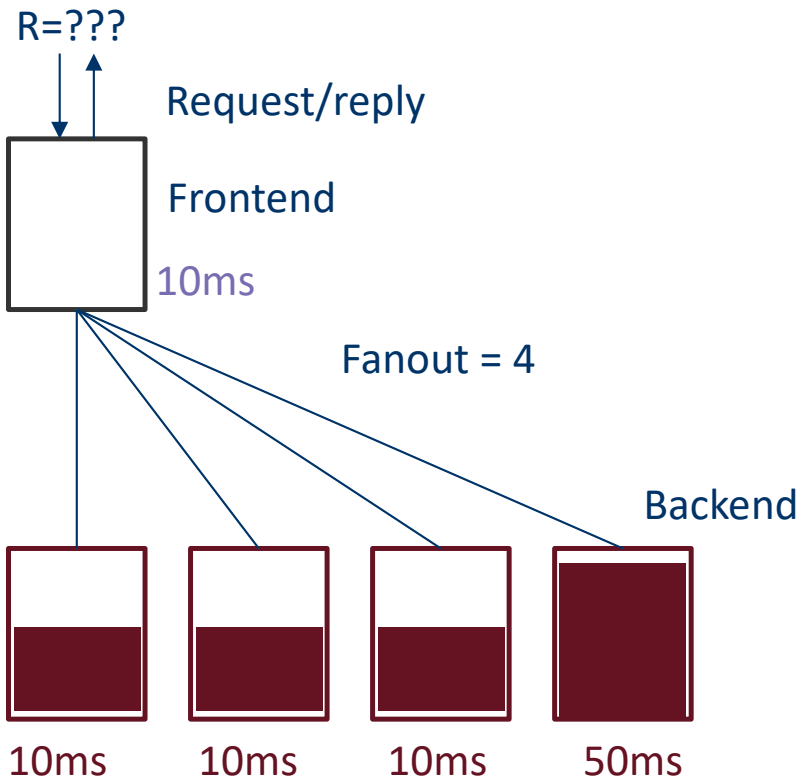
The Tail at Scale

Fanout and stragglers

- What is the response time?
- Gated by **stragglers**
- How to avoid stragglers?



Message latency and per-server R are both variable. **Your mileage may vary**.



Option: quit when it's "good enough"

Table 1. Individual-leaf-request finishing times for a large fan-out service tree (measured from root node of the tree).

	50%ile latency	95%ile latency	99%ile latency
One random leaf finishes	1ms	5ms	10ms
95% of all leaf requests finish	12ms	32ms	70ms
100% of all leaf requests finish	40ms	87ms	140ms

Example: search query results have high utility even if they miss some answers, e.g., because not all shard searches completed.

Source: The Tail at Scale

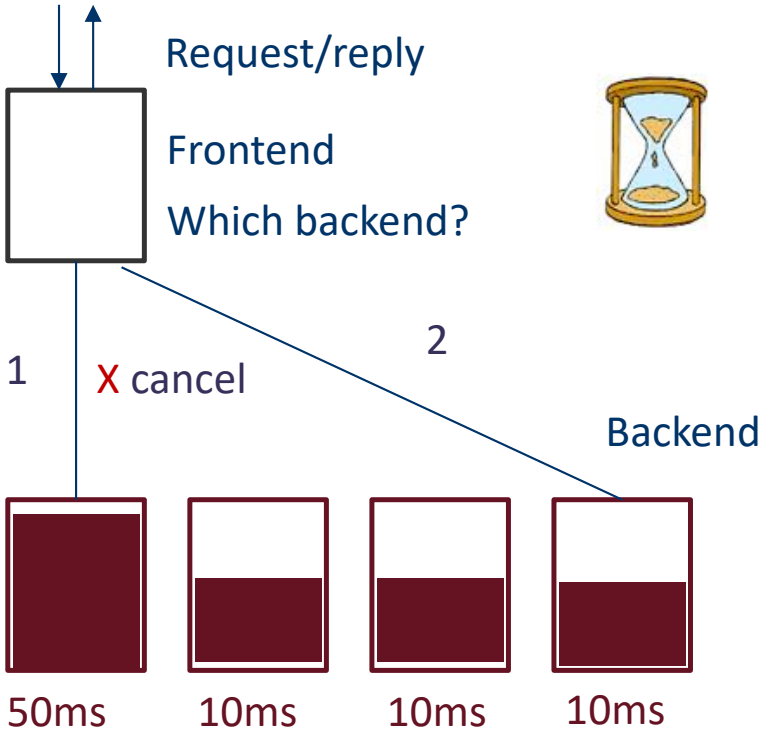
Hedged requests

Suppose:

- Costly backend requests
- Multiple backends

Approach: if a backend is slow, cancel request and try another backend.

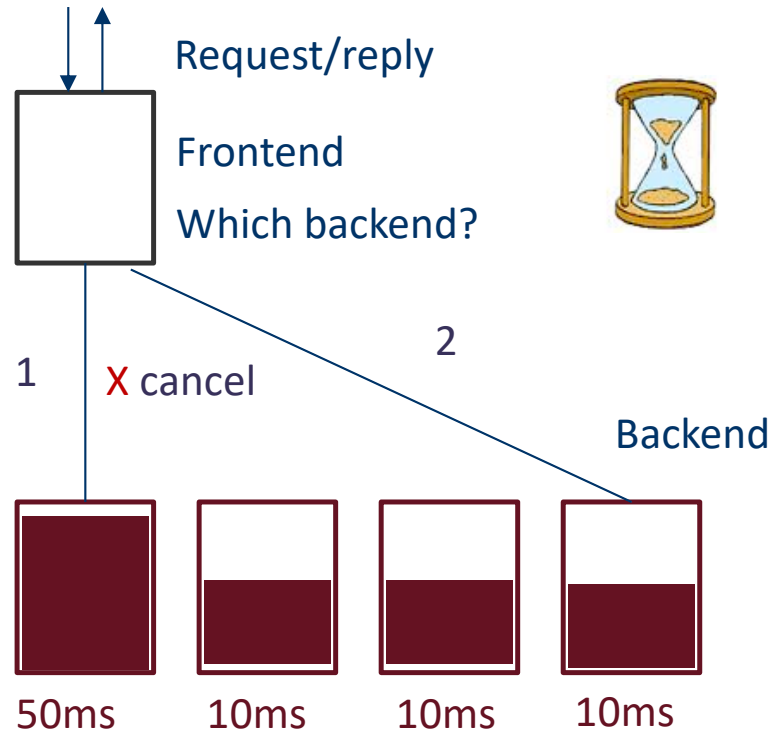
- Optimistic…to a point.
- Slow may be very slow, so find another way.
- Still a good chance to avoid duplicate work.



Tied requests

Approach:

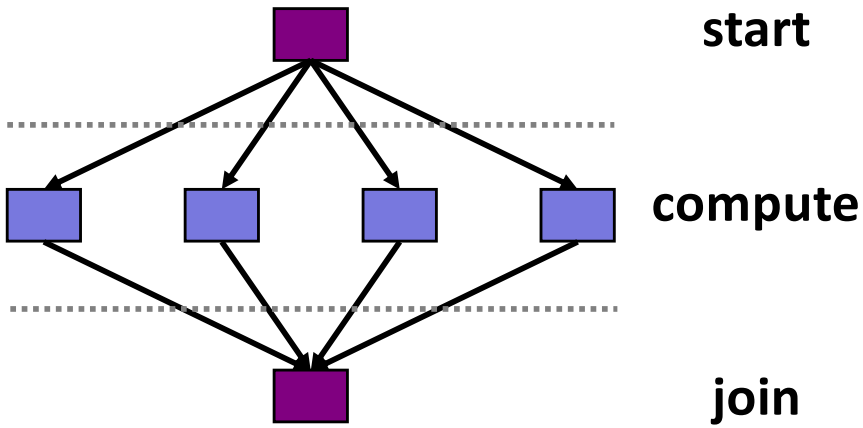
- Send to multiple servers, but tell each about the others.
- Server notifies others when it starts the request task.
- Likely to suppress redundant work.
- "Whoever gets to it first."



Workflow DAGs

Consider a group of tasks to execute, with:

- Task **precedence** relationships: tasks are partially ordered.
- Task **dependencies**: launch only after all predecessors complete.
  - We've seen it before with **make**.
- Total runtime is called "**makespan**".
- What can we parallelize?
- How much can we speed this up?



This is a **directed acyclic graph (DAG)** specifying a **partial order**.

MapReduce: Example

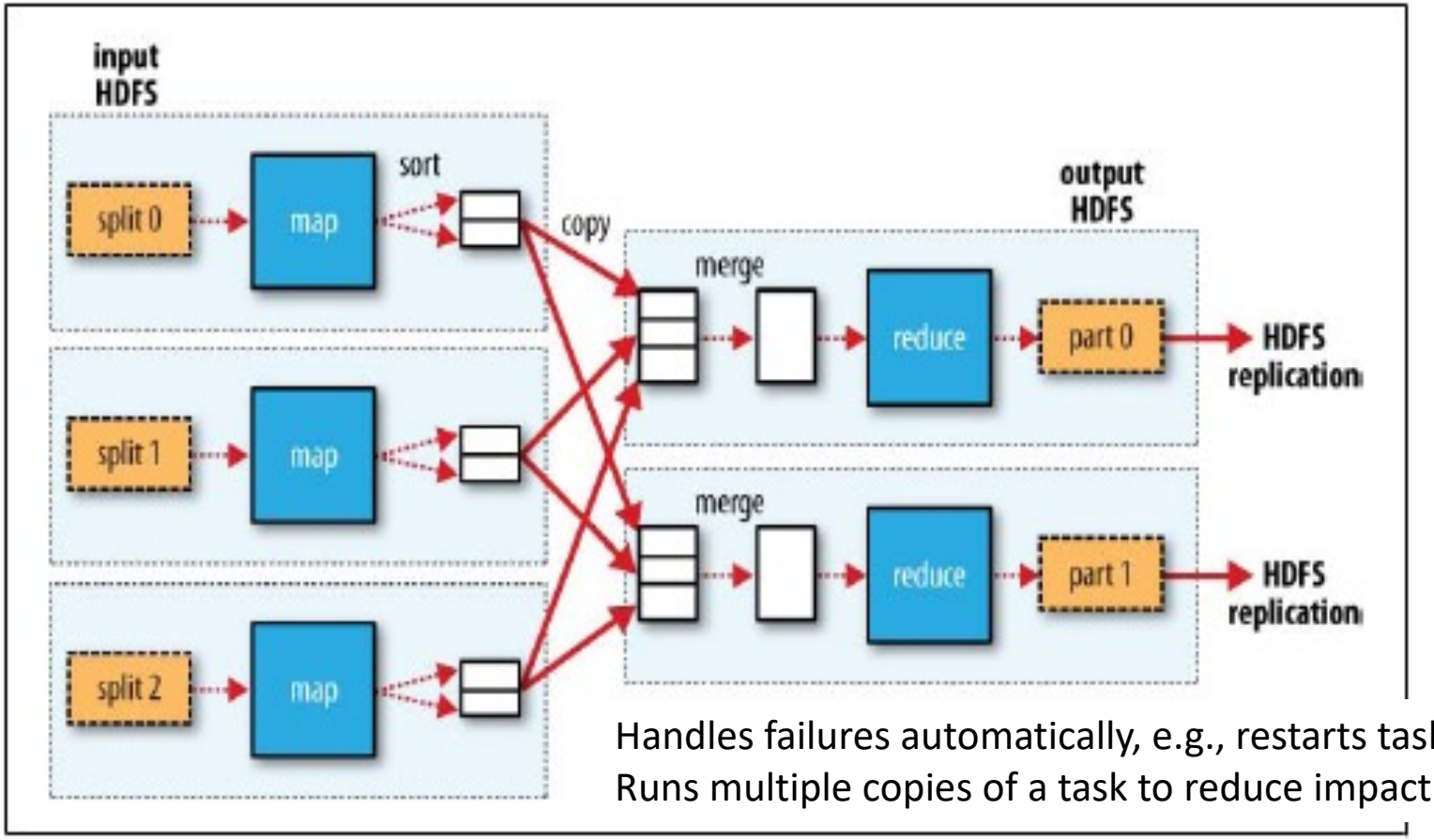


Figure 2-3. MapReduce data flow with multiple reduce tasks

HDFS ≙ GFS

Handles failures automatically, e.g., restarts tasks if a node fails;
Runs multiple copies of a task to reduce impact of stragglers.

Map and Reduce Example

```
function map(String name, String document):
  // name: document name
  // document: document contents
  for each word w in document:
    emit (w, 1)

function reduce(String word, Iterator partialCounts):
  // word: a word
  // partialCounts: a list of aggregated partial counts
  sum = 0
  for each pc in partialCounts:
    sum += pc
  emit sum
```

Why is this interface good?

- The program does not say how to parallelize the computation.
- The program does not say how to facilitate communication between different machines.
- The program does not say how the input is partitioned or how the output should be partitioned.
- The program does not say how failure should be handled.

Map and Reduce functions

- map: (key1, value1) $\rightarrow$ list(key2, value2)
- reduce: (key2, list(value2)) $\rightarrow$ list(value3)

- key2 is called an intermediate key

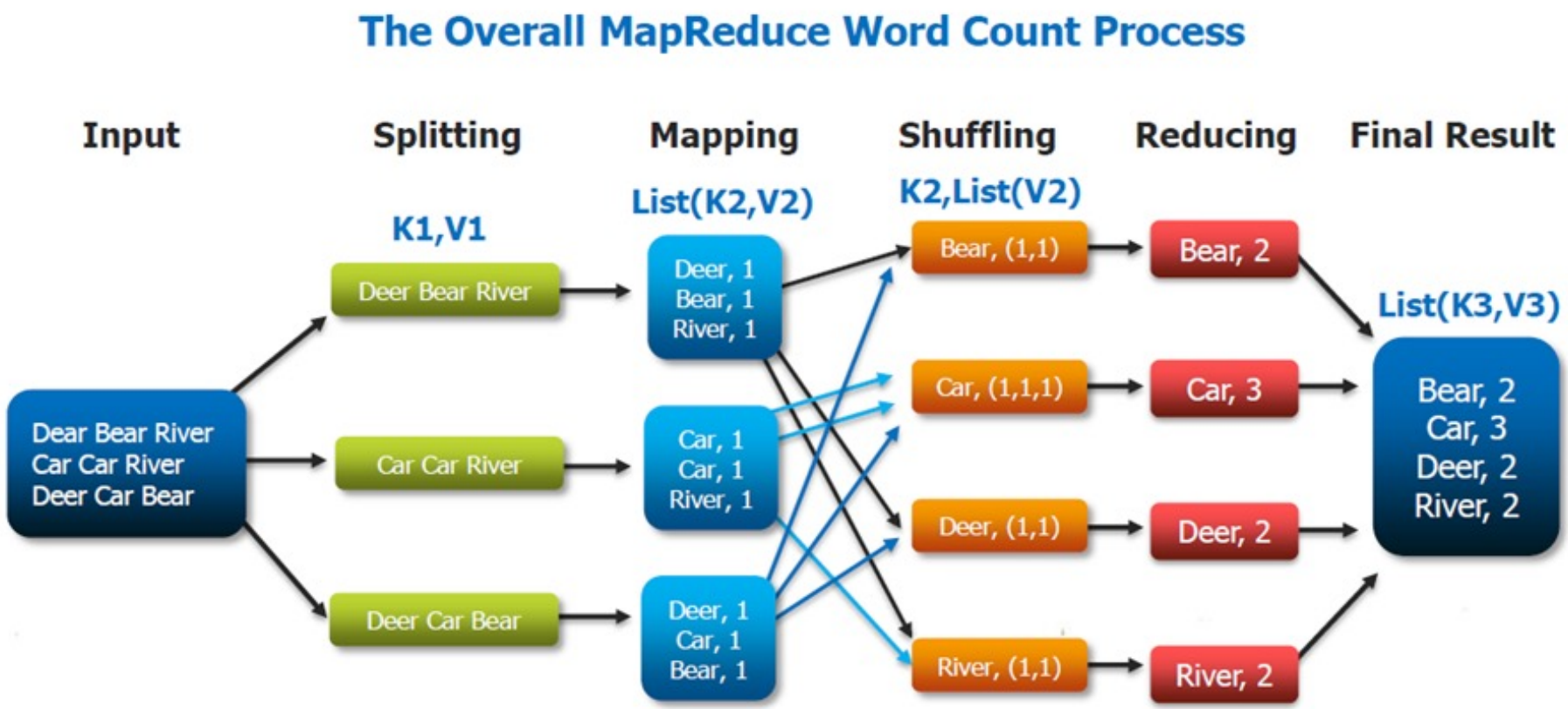
In our example:

key1 is document name (not used), value1 is document contents
key2 is a word, value2 is a count (always 1)
value3 is a count (list always has length 1)

Examples

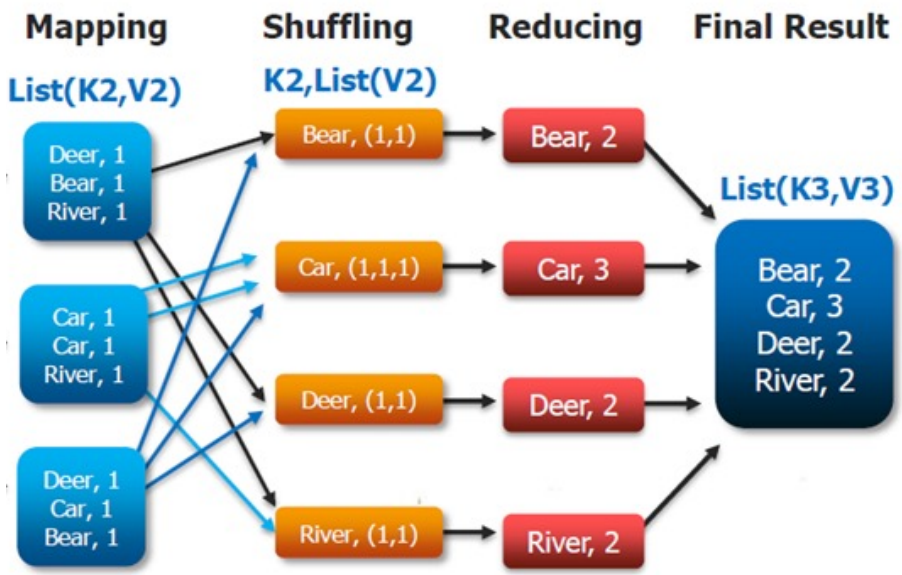
- Inverted index: Find all the webpages that contain each keyword. Map outputs list(<keyword, URL>)
- Reverse web-link graph: for a webpage, find all the webpages pointing to it. Map outputs list(<target URL, source URL>)
- Distributed grep: pattern matching across a large set of files. Map outputs lines matching pattern. Reduce is the identity function.

MapReduce


The Overall MapReduce Word Count Process
Input
Splitting
Mapping
Shuffling
Reducing
Final Result
K1,V1
List(K2,V2)
K2,List(V2)
List(K3,V3)
Dear Bear River
Car Car River
Deer Car Bear
Deer Bear River
Car Car River
Deer Car Bear
Deer, 1
Bear, 1
River, 1
Car, 1
Car, 1
River, 1
Deer, 1
Car, 1
Bear, 1
Bear, (1,1)
Car, (1,1,1)
Deer, (1,1)
River, (1,1)
Bear, 2
Car, 3
Deer, 2
River, 2
Bear, 2
Car, 3
Deer, 2
River, 2

Shuffles and stragglers

- Reduce cannot start until all maps are finished.
- **Shuffle**: map output is sorted and partitioned across reducers.
- Shuffle involves a synchronous barrier (**groupByKey**)
- Wait for the slowest task.
- Mitigations?



MapReduce: Limitations

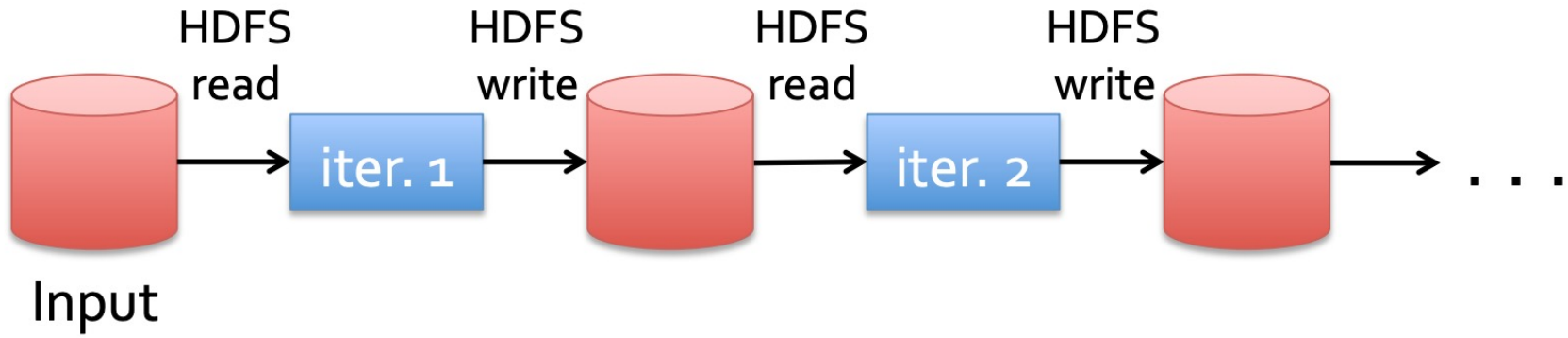
- In 2004, Google tried to optimize for network usage
  - Technique: schedule mappers close to the input data
  - Design exploits "embarrassing parallelism"
    - No communication during map phase
    - No communication during reduce phase
    - Only data exchange in-between

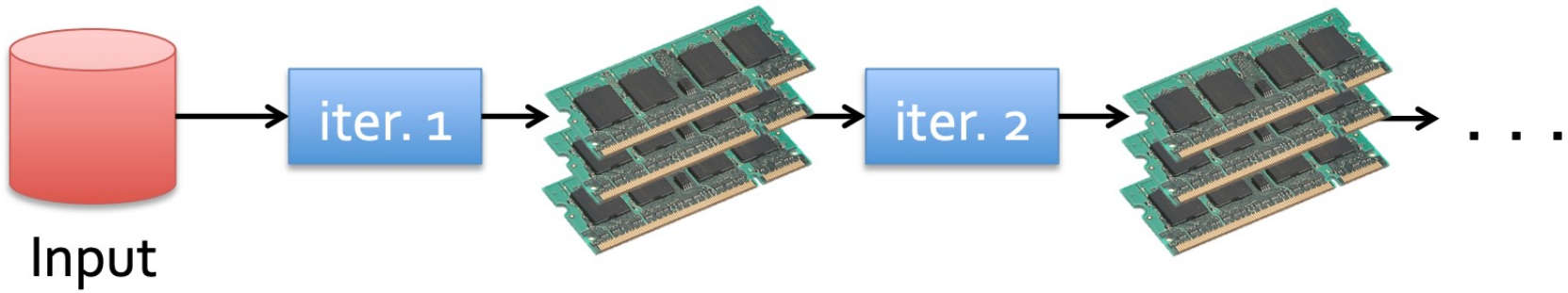
- Why should intermediate data be stored on disks?
  - Spark
- Iterative, stateful computation on matrices/tensors?
  - TensorFlow

Spark

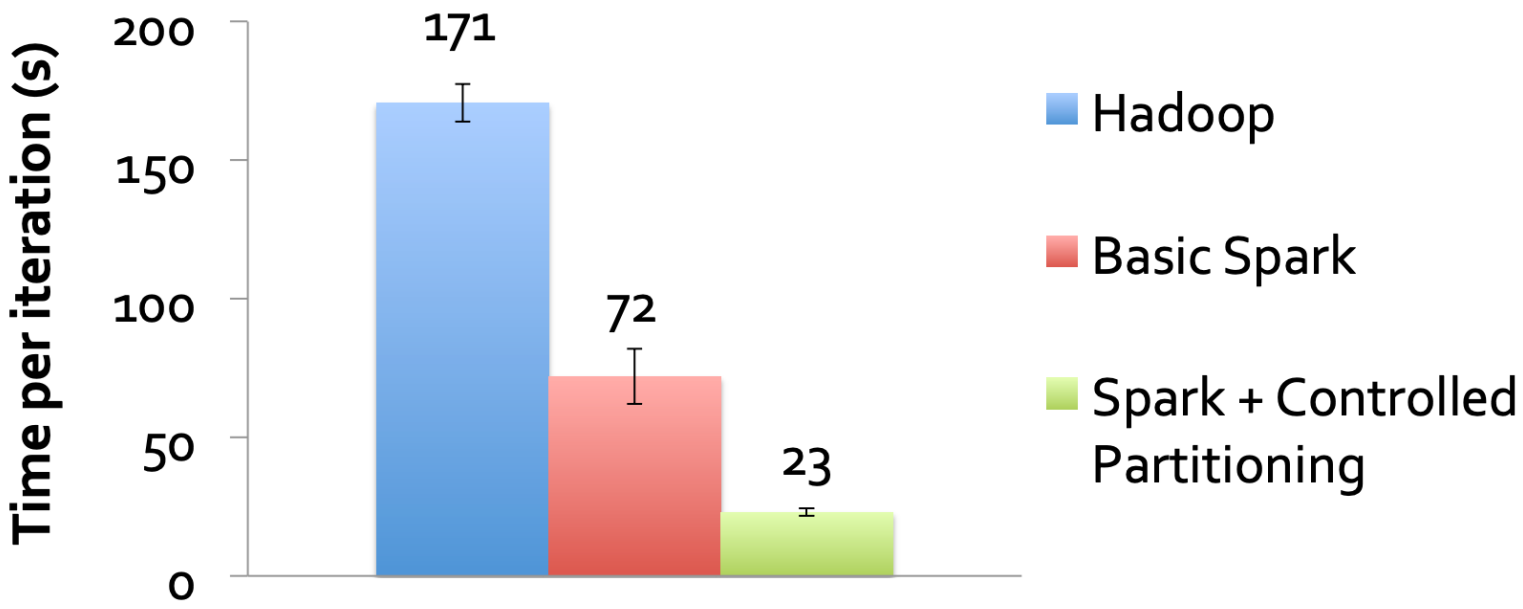
- Generalization of MapReduce primitives, with better support for
  - Iterative computations (e.g., PageRank, K-means clustering, logistic regression)
  - Interactive queries (e.g., data mining tools)
- Higher performance than MapReduce
  - Keep intermediate results in memory

MapReduce vs. Spark





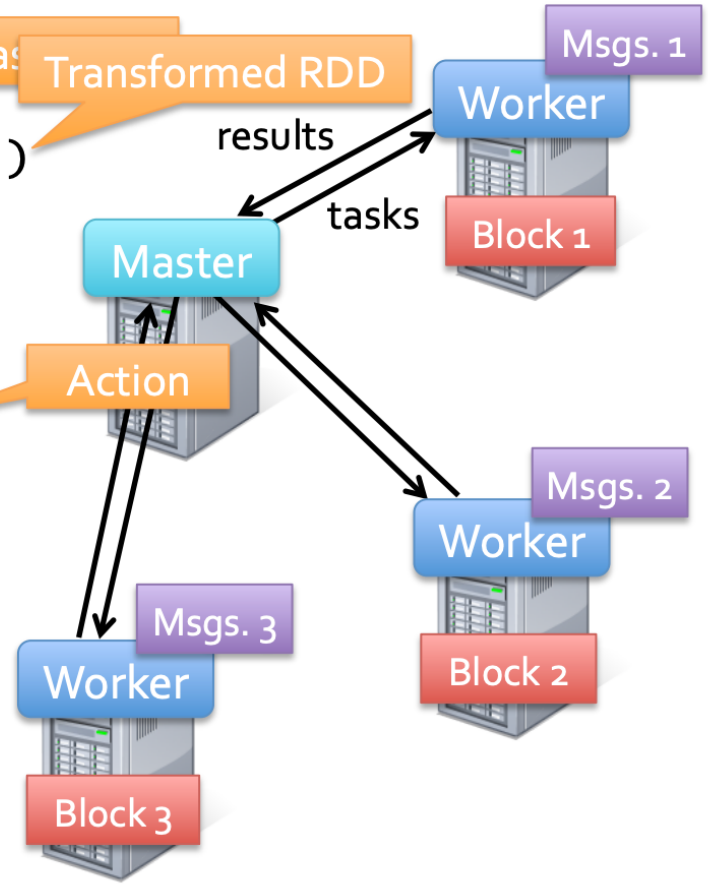
Performance of PageRank on Hadoop vs. Spark



Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))
messages.persist()

messages.filter(_.contains("foo")).count
messages.filter(_.contains("bar")).count
```



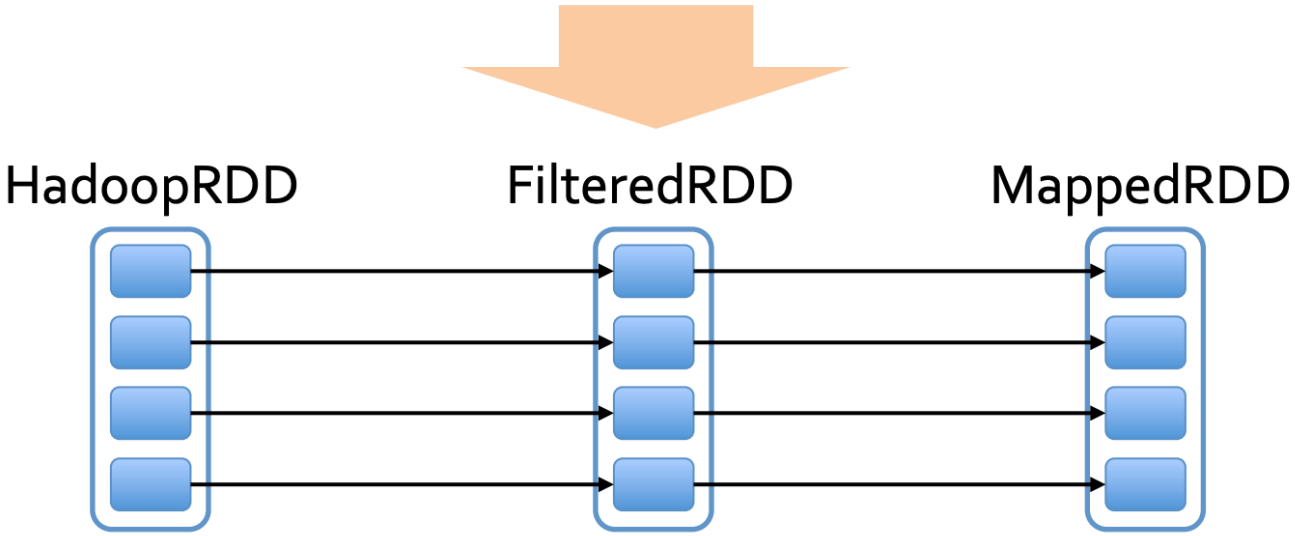
Result: scaled to 1 TB data in 5-7 sec (vs 170 sec for on-disk data)

Resilient Distributed Datasets (RDD)

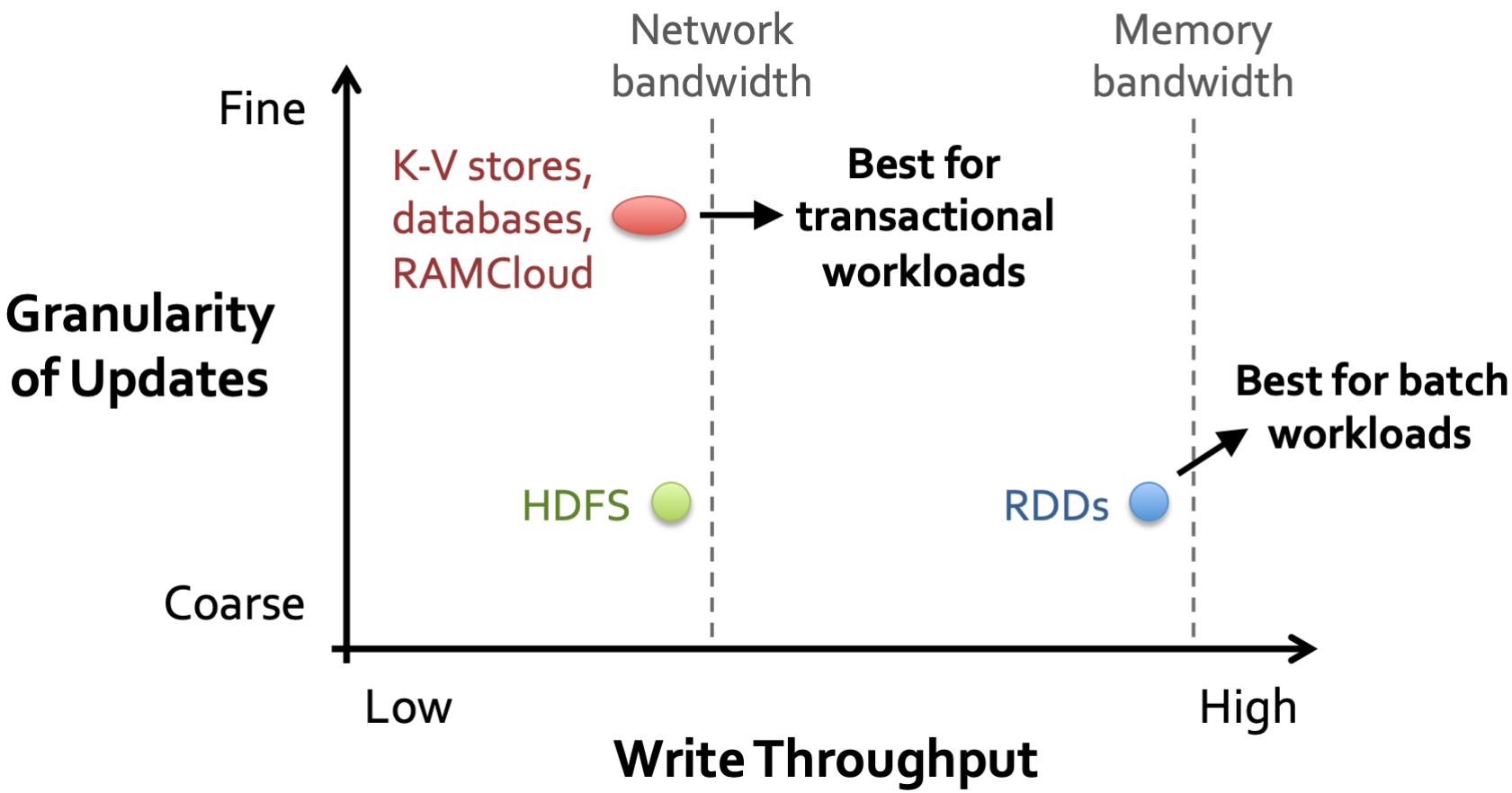
- A collection of records, e.g., [(K, V)] or [V]
- Stored in-memory, partitioned across worker machines
- Immutable, and therefore easier to reconstruct on failure
- Can compute partitions redundantly to mitigate stragglers
- Initialized by reading from file system (HDFS)
- Or created as output of coarse-grained deterministic "transformations", e.g., map or filter
- Can be queried through "actions", e.g., count

RDDs track the graph of transformations that built them (their *lineage*) to rebuild lost data

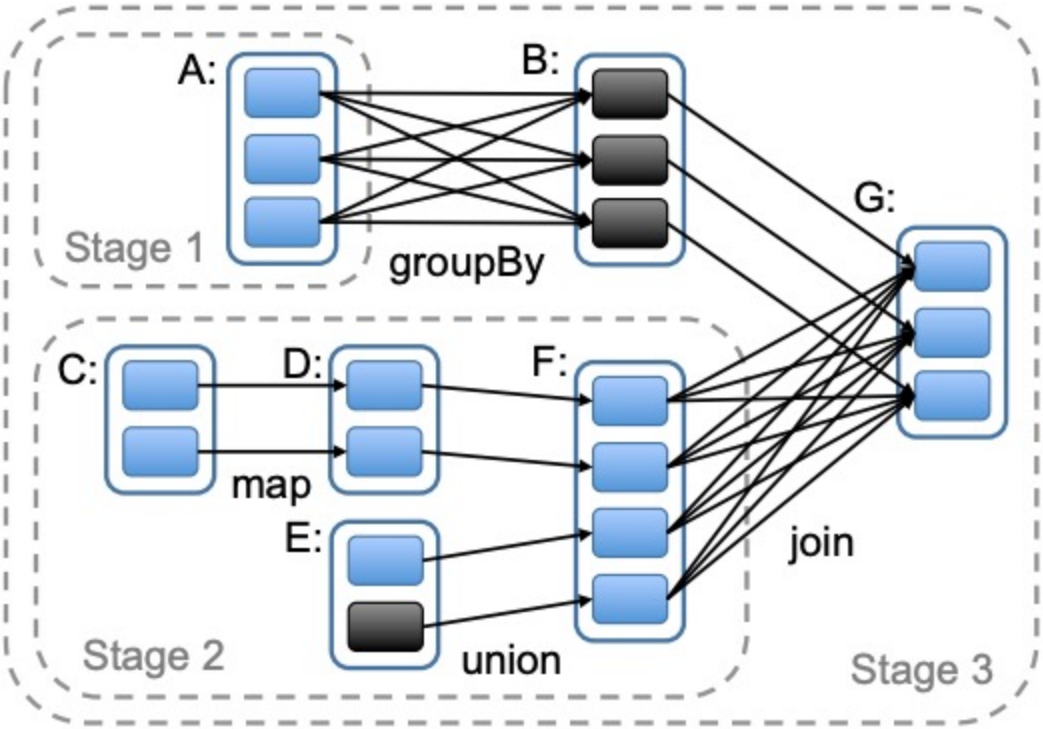
E.g.:
```
messages = textFile(...).filter(_.contains("error"))
                         .map(_.split('\t')(2))
```



Tradeoff


Network bandwidth
Memory bandwidth
Fine
Coarse
Granularity of Updates
K-V stores, databases, RAMCloud
Best for transactional workloads
Best for batch workloads
HDFS
RDDs
Low
High
Write Throughput

Example of how Spark computes job stages



Boxes with solid outlines (e.g., A): RDDs.

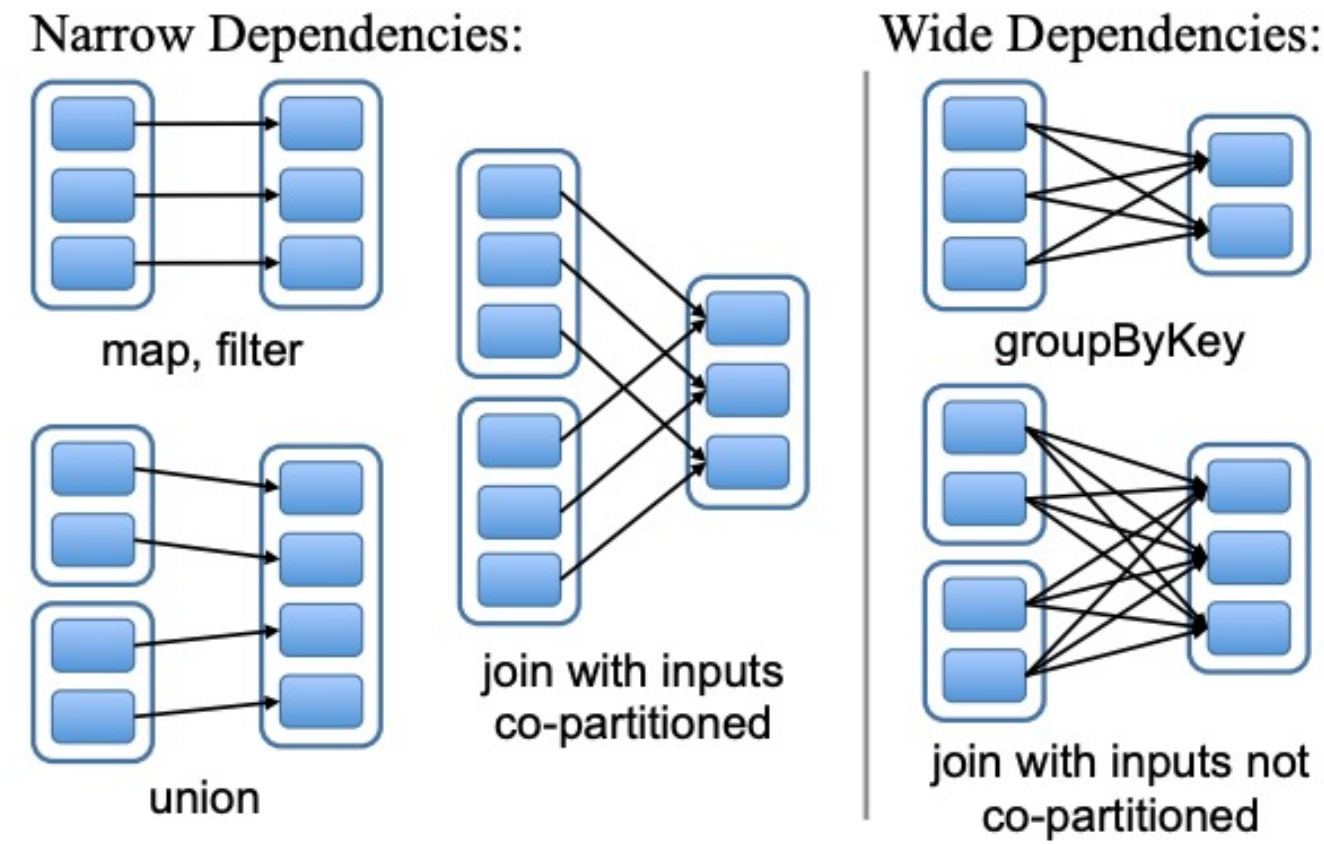
Partitions are shaded rectangles (e.g., A has 3 partitions). Black if already in memory.

Each stage contains as many pipelined transformations with narrow dependencies as possible.

In this example Stage 1's output is already in memory, so to compute G, run Stage 2 then Stage 3.

Narrow vs. wide dependencies

Default: hash partitions (unsorted)



Global rehash and/or sort: Shuffle barrier again!

Spark transformations and actions

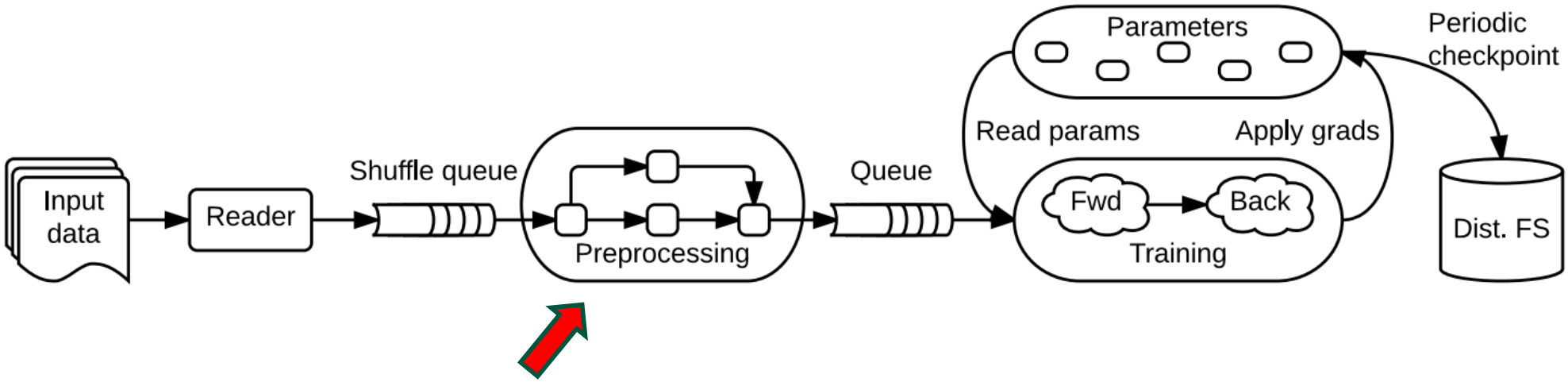
Transformations	$map(f : T \Rightarrow U)$	:	$RDD[T] \Rightarrow RDD[U]$
	$filter(f : T \Rightarrow Bool)$	:	$RDD[T] \Rightarrow RDD[T]$
	$flatMap(f : T \Rightarrow Seq[U])$	:	$RDD[T] \Rightarrow RDD[U]$
	$sample(fraction : Float)$	:	$RDD[T] \Rightarrow RDD[T]$ (Deterministic sampling)
	$groupByKey()$	:	$RDD[(K, V)] \Rightarrow RDD[(K, Seq[V])]$
	$reduceByKey(f : (V, V) \Rightarrow V)$	:	$RDD[(K, V)] \Rightarrow RDD[(K, V)]$
	$union()$	:	$(RDD[T], RDD[T]) \Rightarrow RDD[T]$
	$join()$	:	$(RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (V, W))]$
	$cogroup()$	:	$(RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (Seq[V], Seq[W]))]$
	$crossProduct()$	:	$(RDD[T], RDD[U]) \Rightarrow RDD[(T, U)]$
	$mapValues(f : V \Rightarrow W)$	:	$RDD[(K, V)] \Rightarrow RDD[(K, W)]$ (Preserves partitioning)
	$sort(c : Comparator[K])$	:	$RDD[(K, V)] \Rightarrow RDD[(K, V)]$
	$partitionBy(p : Partitioner[K])$	:	$RDD[(K, V)] \Rightarrow RDD[(K, V)]$
Actions	$count()$	:	$RDD[T] \Rightarrow Long$
	$collect()$	:	$RDD[T] \Rightarrow Seq[T]$
	$reduce(f : (T, T) \Rightarrow T)$	:	$RDD[T] \Rightarrow T$
	$lookup(k : K)$	:	$RDD[(K, V)] \Rightarrow Seq[V]$ (On hash/range partitioned RDDs)
	$save(path : String)$	:	Outputs RDD to a storage system, *e.g.*, HDFS

Note that Spark function names are chosen to match Scala APIs and differ from MapReduce.

TensorFlow goal and features

- **Support training and inference of large-scale machine-learning models**
- Represents network computation and shared state as dataflow graphs
- Training algorithms are also expressed as dataflow graphs
- Automatically maps nodes of the dataflow graph across servers, or across CPUs, GPUs, or TPUs
- But also runs on a laptop or mobile phone

Dataflow graph for a training pipeline



e.g., decode images, apply random distortions

Why not use MapReduce or Spark?

- Models are extremely large, e.g.,
  - GPT-5 is estimated to have trillions of parameters
  - OpenAI used approximately 200,000 GPUs to train GPT-5 on trillions of tokens
  - Training cost hundreds of millions of dollars

- To reduce cost, training has to take advantage of specialized hardware (e.g., GPUs and TPUs) and highly optimized kernels (implementations) of common operations